Troubleshooting Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> From the client's use-case list (Yusuf), item 5, Troubleshooting Automation: "Troubleshooting page for issue moment issue landed on SNOW acknowledgement mail with the asking details for primary troubleshooting and working accordingly by suggesting troubleshooting steps using CLI command. And create template on basis of reason, action and next action." The list closes with: "Our Aim is to do automation using Claude for all these content."

Load balancer and data center operations teams spend the first part of every incident on the same mechanical work: acknowledge the ticket, ask the caller for the VIP, start time, scope and last change, log in to the device and type the same handful of show commands, then write a resolution note. The client's request describes exactly this loop, from the moment an issue lands in ServiceNow to a reason, action and next action template, and asks for it to be automated with Claude.

The value is not in letting an agent change devices. It is in removing the repetitive reading and writing around an incident while keeping engineers in control. ServiceNow already provides the hooks for this: a Table API for reading and updating incidents [1], watermarked notifications that route replies back to the record [2], Flow Designer triggers [3] and KCS-style knowledge templates [7]. Claude Code, the Agent SDK and MCP provide scoped tool access, structured output, permission rules and hooks that can enforce a read-only default [8][10][11][14].

The course therefore teaches a supervised pattern: deterministic collectors gather typed evidence from F5, NetScaler, Radware, other load balancers and Cisco ACI; Claude ranks causes and proposes cited CLI steps; a person approves anything that changes state; and every action is logged in an audit trail and in ServiceNow work notes. Learners finish with a working troubleshooting page and a runbook they can adapt to their own estate.

2. How the outline was shaped

Decision	Rationale
Keep the client's exact sequence as the course spine: SNOW intake, acknowledgement with asking details, CLI troubleshooting steps, reason/action/next action template, troubleshooting page.	The client input is one sentence describing a workflow; following it in order makes each module map to a step the client's engineers already recognize.
The agent drafts the acknowledgement mail and resolution note; a person sends and resolves.	Customer communication and ticket closure carry accountability; the MCP specification itself calls for a human able to deny tool invocations [16].
Read-only by default, with state-changing commands behind an approval gate enforced by deny rules and a PreToolUse hook rather than by prompt instructions.	CLAUDE.md is context, not enforcement [12]; deny-first rules and blocking hooks are the documented enforcement points [10][11].
Deterministic collectors produce typed evidence; Claude only interprets and cites it.	Separating collection from reasoning makes every claim in the final template traceable to raw device output and keeps CLI parsing testable.
Cover F5, NetScaler, Radware Alteon, 'Anaya LB' and Cisco ACI, using free trials, VPX Express, the DevNet ACI simulator and recorded fixtures.	The client's other use cases name these platforms; free and trial options keep the lab affordable [27][29][30], and recorded fixtures cover platforms without lab access.
End with a FastAPI page, pytest regression suite and GitHub Actions.	The client asked for a 'troubleshooting page'; tests and CI make the automation maintainable after the course.

3. Audience and prerequisites

Gemnoo Tech TRAINING UNIT For more info - https://gemnoo.com

Role	What they need	Starting point
Load balancer and network operations engineer (L2/L3)	Faster first response and less repetitive typing during incidents without losing control of devices.	Fluent in tmsh or NetScaler CLI, some Python, limited API experience.
NOC and service desk analyst	Consistent acknowledgement mails and complete tickets before hand-off.	Strong ServiceNow user, little device or scripting experience; pairs with an engineer in device labs.
Data center / ACI engineer	Quick fault and health lookups tied to the incident.	Knows APIC GUI, may not have used the REST API.
IT automation lead	A governed pattern for agent use in operations: permissions, approvals, audit and CI.	Python and Git experienced, may not know load balancer specifics.

Prerequisites from the outline

- Basic networking and load balancer concepts (VIP, pool, HA)
- Comfort with a CLI such as F5 tmsh or NetScaler shell
- Basic Python and Git
- Familiarity with ServiceNow incidents is helpful, not required

Checks before enrolment

- Can you explain VIP, pool, pool member, health monitor and HA pair in your own words?
- Can you log in to an F5 or NetScaler and run a show command for a virtual server?
- Can you read and edit a 50-line Python script and run it in a virtual environment?
- Can you clone a Git repository, create a branch and open a pull request?
- Do you have (or can you create) a ServiceNow Personal Developer Instance account?
- Can you make a REST call with curl or Postman and read a JSON response?

Course outcomes

- Explain the incident-to-resolution workflow in ServiceNow and where an agent can and cannot act
- Set up Claude Code, the Claude Agent SDK and MCP servers scoped to incident triage with read-only tools by default
- Pull incidents through the ServiceNow Table API, draft the acknowledgement mail and ask for the primary troubleshooting details that are missing
- Collect typed evidence from F5, NetScaler, Radware, other load balancers (Anaya LB) and Cisco ACI and have Claude suggest ranked CLI troubleshooting steps as structured JSON
- Enforce read-only versus state-changing command classes, human approval and a full audit trail into ServiceNow work notes

GenNoor Tech
AI INNOVATE BUILD
For more info, visit: https://gennoor.com

- Generate and post the reason, action and next action template and ship a tested troubleshooting page

4. Lab environment

Each learner gets a ServiceNow PDI with five synthetic incidents, access to shared lab load balancers (F5 BIG-IP VE and a NetScaler VPX HA pair), the Cisco DevNet ACI simulator, and recorded fixtures for Radware Alteon and 'Anaya LB'. Learners work in a course Git repository with Claude Code, the Agent SDK and two read-only MCP servers.

Components

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
ServiceNow Personal Developer Instance	Current release offered by the Developer Site (Zurich family at time of writing)	Incidents, work notes, notifications, Flow Designer and knowledge articles.	Free from the ServiceNow Developer Site.	Reclaimed after ten days without use and hibernates when idle; have learners sign in the day before the course [28]. Activate the Knowledge Management Advanced plugin for KCS templates [7].
F5 BIG-IP Virtual Edition	17.x (matches the TMSH 17.x reference) [4]	Primary load balancer for tmsh and iControl REST labs.	30-day free trial keys from MyF5 [29]; a paid VE lab license is an alternative for repeated cohorts.	Trial keys expire; plan provisioning within 30 days of delivery.
NetScaler VPX Express (HA pair)	14.1	NITRO API, show ha node and HA sync scenarios.	VPX Express needs no license file (20 Mbps, 250 SSL sessions) [30].	Two VPX instances on the same subnet form the HA pair.
Cisco ACI Simulator	DevNet sandbox version (shared always-on or reservable)	APIC REST queries for faults, health and endpoints.	Cisco DevNet always-on sandbox (no reservation) or reservable sandbox up to 5 days [27].	The always-on instance is shared and read-mostly; use the reservable one if labs need fixed data.
Radware Alteon	31.x documentation set	Alteon CLI evidence collection.	Alteon VA if the client can provide a license; otherwise recorded CLI fixtures [25][26].	Obtain the Command Reference and REST API User Guide for the matching version from the Radware portal [26].
'Anaya LB' (client-named load balancer)	Unknown	Generic CLI capture adapter exercise.	Recorded, sanitized output supplied by the client.	No public documentation was found under this name; confirm the actual product with the client.
Claude Code and Claude Agent SDK (Python)	Current releases	Interactive work, triage service, MCP integration.	Install Claude Code and the claude-agent-sdk Python package; API	Use project-scope .mcp.json and settings.json from the course repository [8][10].

			key per learner or team.	
Python toolchain	Python 3.11+, Pydantic 2, FastAPI, pytest, httpx, MCP Python SDK	Collectors, MCP servers, page and tests.	Course requirements file.	The MCP Python quickstart uses MCPServer and mcp.run(transport='stdio') [17].
GitHub repository with Actions	GitHub.com or GitHub Enterprise	Pull requests, CI and prompt regression checks.	Course organization with one repository per team.	Protect main with required reviews and status checks.

Fixtures and sample data

Fixture	Contents	Used in
Synthetic incident pack	Five incidents (VIP down, pool member flapping, HA sync failure, certificate expiry, slow application) with deliberately missing details.	The troubleshooting workflow and the ServiceNow incident lifecycle; Incident intake through the Table API and the acknowledgement mail
Recorded device output	tmsh, iControl REST, NITRO, show ha node, Alteon /stats/slb and APIC JSON responses for each incident, sanitized.	Evidence collection from load balancers and ACI; The troubleshooting page: service, tests and delivery
Bad-evidence variants	Missing, stale and contradictory evidence bundles for three incidents.	Claude suggests troubleshooting steps as structured JSON
Allowlist and hook pack	allowlist.yaml, PreToolUse hook script, settings.json.	Guardrails, approvals and audit for incident automation
Capstone incident	An unseen HA sync failure incident with matching device state.	Capstone: incident to resolution note, end to end

Lab safety rules

- Never connect lab agents to production devices or the production ServiceNow instance.
- Collectors use read-only accounts; state-changing commands run only on lab devices and only with an approval record.
- No credentials in prompts, transcripts, fixtures or the repository; use ${VAR} expansion and repository secrets [8].
- Never run the triage agent in bypassPermissions mode [15].
- Sanitize client-supplied fixtures (hostnames, IPs, user names) before they enter the repository.

5. Module-by-module content plan

Day 1

Module 1: The troubleshooting workflow and the ServiceNow incident lifecycle

Goal. Learners map today's manual incident-to-resolution workflow for load balancer and ACI incidents and mark exactly where an agent may read, draft and explain, and where a person must act.

Outline topics.

- Incident states, assignment groups, acknowledgement mail and the first troubleshooting reply as they happen today
- Mapping the manual workflow: ticket lands, details requested, evidence gathered, steps suggested, resolution note written
- Where an agent may act (read, draft, explain) and where it may not (change a device, close a ticket) without a person
- The lab: a ServiceNow developer instance, lab load balancers and a Cisco ACI sandbox

Learning objectives.

- Learners can describe the incident states and the hand-offs between service desk, assignment group and resolver.
- Learners can explain how ServiceNow ties an emailed reply back to the incident record.
- Learners can draw the manual workflow from ticket creation to resolution note and label every decision point.
- Learners can classify each workflow step as agent-allowed (read, draft, explain) or person-only (change a device, send mail, close a ticket).
- Learners can name the lab systems and what each one stands in for in production.

The incident record and its lifecycle

- Incident records live in the incident table and are read and updated through /api/now/table/incident and /api/now/table/incident/{sys_id} [1].
- Walk through the typical state progression used in most instances (New, In Progress, On Hold, Resolved, Closed) and tell learners to confirm the actual choice values in their own instance's state dictionary, because instances are customized.
- Assignment group, assigned_to, priority, short_description, description, cmdb_ci and work_notes are the fields the troubleshooting workflow reads and writes; work_notes appear as a field on Table API responses [1].
- Close notes and close code are written at resolution; in this course they are only ever drafted by the agent and submitted by a person.
- Every update the agent proposes must be traceable to an engineer action in the audit trail (built in the guardrails module).

Acknowledgement mail and how replies come back

- Outbound notifications carry a watermark at the bottom so the instance can match an incoming reply to the existing record [2].
- Inbound email actions script what the instance does with that reply, for example append it to the incident as a comment [2].
- Digests can summarize frequent updates into one mail, which matters when a P2 is being updated every few minutes [2].
- The 'acknowledgement mail with asking details' from the client input becomes a drafted reply in this course; the agent never sends mail itself.
- Show a real-style acknowledgement mail and mark which sentences are boilerplate and which are the primary troubleshooting questions.

Mapping the manual workflow

- Steps: ticket lands, engineer acknowledges, asks for missing details, gathers evidence on the device, runs first checks, suggests or performs a fix, writes the resolution note.
- For each step record the inputs (ticket fields, device output), the decision made, who made it and how long it took.
- Highlight the repeated first five commands an on-call engineer types for a VIP-down ticket; these become the read-only collectors on day 2.
- Capture the reason, action and next action fields the client wants in the final template from the start, so the evidence chain is designed backwards from the note.
- Record hand-offs (service desk to lb-ops, lb-ops to app team) because they become the 'next action owner' field.

Automation boundaries

- Agent may: read the incident, read device state through allowlisted read-only commands, draft the acknowledgement, propose ranked steps, draft the resolution note.
- Agent may not without a person: send mail, run any state-changing command, change incident state to Resolved or Closed, publish a knowledge article.
- MCP's own specification says there should always be a human in the loop able to deny tool invocations [16]; the course turns that into an enforced control, not a convention.
- Write the boundaries as a one-page policy that later becomes permission rules and hooks [10] [11].
- Agree a stop rule: if the agent cannot cite evidence for a claim, the claim is marked unknown.

The lab estate

- ServiceNow Personal Developer Instance: free, reclaimed after ten days without use, hibernates when idle [28].
- F5 BIG-IP VE on a 30-day trial key from MyF5 [29] and NetScaler VPX Express with no license file, 20 Mbps [30].
- Cisco DevNet always-on ACI Simulator (shared, no reservation) or the reservable simulator for up to five days [27].

- Radware Alteon and 'Anaya LB' are represented by recorded CLI fixtures unless the client provides a lab appliance (see lab environment).
- Synthetic incidents with real symptoms are loaded into the PDI on day 1 so every later lab uses the same tickets.

Instructor demo.

1. Open a synthetic P2 incident INC0048211 'app portal slow / timeouts' in the PDI and show its fields through the list view and through GET /api/now/table/incident?sysparm_query=number=INC0048211 [1].
2. Show the acknowledgement notification and point out the watermark line that links replies to the record [2].
3. Reply to the mail from a test mailbox and show the reply landing on the incident through the inbound email action [2].
4. Log in to the lab F5 and type the usual first checks by hand (tmsh show ltm virtual, tmsh show ltm pool) while narrating each decision [4].
5. Write the resolution note by hand in reason, action, next action form.
6. Draw the workflow on the whiteboard and color each step green (agent may act), amber (agent drafts, person sends) or red (person only).

Hands-on lab. Walk one real-style incident from creation to resolution by hand and record every decision an engineer makes.

1. Log in to your PDI and confirm the five synthetic incidents are present in the lb-ops assignment group.
2. Pick the VIP-down incident and read every field; list the ones you would use to start troubleshooting.
3. Write the acknowledgement mail you would send today, including the questions you would ask the caller.
4. Connect to the lab F5 or NetScaler and run the read-only commands you would normally run; paste each command and output into a worksheet with timestamps.
5. Record each decision you made, what evidence drove it and how long it took.
6. Write the resolution note in reason, action, next action form.
7. Color each step on the workflow template green, amber or red and justify the color in one sentence.
8. Compare your map with a partner and resolve any step you colored differently.

Lab success criteria.

- The workflow map covers ticket creation to resolution note with no missing step.
- Every step has an owner and a green, amber or red classification with a reason.
- Every command in the worksheet is marked read-only or state-changing.
- The resolution note separates reason, action and next action.

Deliverable. Annotated workflow map and automation boundaries.

Common pitfalls.

- Assuming the instance uses out-of-box state values; always read the state choices from the instance.
- Letting the agent send the acknowledgement mail 'because it is only an acknowledgement'; mail to customers is a person-only action in this course.
- Mapping the ideal process instead of what engineers actually do at nine at night.
- Forgetting that a PDI is reclaimed after ten days without sign-in; learners lose their data between cohorts [28].

Knowledge check.

1. How does ServiceNow match an emailed reply to the original incident? Answer: Through the watermark label added to outbound notifications, processed by an inbound email action [2].
2. Name two actions the agent may take on its own and two it may never take without a person. Answer: May: read the incident, run allowlisted read-only commands, draft mail and notes. May not: send mail, run state-changing commands, resolve or close the ticket, publish a KB article.
3. Which Table API path updates a single incident? Answer: PATCH or PUT /api/now/table/incident/{sys_id} [1].
4. Why is the workflow designed backwards from the resolution note? Answer: So that every reason and action line in the final template has evidence collected for it earlier in the flow.

Slides and assets to prepare.

- Workflow map template with green, amber and red swimlanes.
- Five synthetic incident records (VIP down, pool member flapping, HA sync failure, certificate expiry, slow application) as an import file for the PDI.
- Sample acknowledgement mail with watermark highlighted.
- One-page automation boundary policy template.
- Lab access sheet with PDI, F5, NetScaler and ACI sandbox endpoints.

Sources for this module. 1, 2, 4, 10, 11, 16, 27, 28, 29, 30

Module 2: Setting up Claude Code, the Agent SDK and MCP for incident triage

Goal. Learners build a working agent environment in which Claude can reach only a read-only ServiceNow tool and a read-only device tool, with credentials outside prompts and transcripts.

Outline topics.

- Claude Code project configuration, CLAUDE.md conventions and permission rules for this workflow
- Claude Agent SDK: sessions, tool definitions, structured JSON outputs and retries
- MCP servers for ServiceNow and for device read-only queries, with tool allowlists
- Keeping credentials in environment and vault, never in prompts or transcripts

Learning objectives.

- Learners can write a project CLAUDE.md and permission rules that encode the automation boundaries from module 1.
- Learners can build a stdio MCP server in Python and register it with claude mcp add at project scope.
- Learners can define in-process tools with the Agent SDK and restrict which tools are available and auto-approved.
- Learners can keep credentials in environment variables or a vault and reference them from .mcp.json without committing them.

Claude Code project configuration

- Project instructions go in ./CLAUDE.md or ./.claude/CLAUDE.md and are shared through Git; /init generates a starting file [12].
- CLAUDE.md is context, not enforcement; anything that must be blocked goes into permission rules or a PreToolUse hook [12].
- Keep CLAUDE.md short (the docs suggest under 200 lines) and import long runbooks with @path [12].
- Permission rules live in .claude/settings.json (team) and .claude/settings.local.json (personal) [10].
- Rules are evaluated deny, then ask, then allow; first match wins, so deny Bash(ssh *) cannot be undone by a narrower allow [10].

MCP servers for incident triage

- MCP standardizes how Claude reaches tools and data so the same server works from Claude Code and the Agent SDK [9].
- Build servers with the Python SDK: from mcp.server import MCPServer, @mcp.tool(), mcp.run(transport='stdio') [17].
- A stdio server must never print to stdout; send logs to stderr with the logging module [17].
- Register with claude mcp add --transport stdio servicenow-ro -- python servers/snow_ro.py; use --scope project to write .mcp.json for the team [8].
- Tools are discovered with tools/list and invoked with tools/call; return failures with isError true so Claude can react [16].

Claude Agent SDK basics

- The SDK runs the same agent loop as Claude Code from Python or TypeScript; use it for the scripted triage service, and the CLI for interactive work.
- Define in-process tools with @tool and create_sdk_mcp_server, then pass them in ClaudeAgentOptions(mcp_servers={...}) [13].
- Tool names follow mcp__{server}__{tool}; list exactly the read-only ones in allowed_tools [13].
- Use the tools option to remove built-ins the triage agent does not need, such as Bash and Write [13].

- Annotations such as readOnlyHint are metadata only; the handler must actually be read-only [13][16].

Credentials outside prompts

- .mcp.json supports ${VAR} and ${VAR:-default} expansion in command, args, env, url and headers, so secrets stay in the environment [8].
- Unset variables are left as literal text and claude mcp list warns about them; check before running the lab [8].
- Device and ServiceNow credentials are injected into the MCP server process, never returned in tool output.
- Use a dedicated read-only ServiceNow integration user and read-only device accounts (for example an F5 Guest or Operator role) for the collectors.
- Add a deny rule for reading .env and secrets paths so they cannot be pulled into a transcript [10].

Instructor demo.

1. Run /init in the course repository and edit CLAUDE.md to state the boundaries from module 1 [12].
2. Add .claude/settings.json with deny rules for Bash(ssh *), Bash(curl *) and Read(./.env) and show the deny-first behavior with a test command [10].
3. Write a 30-line snow_ro.py MCP server with get_incident and get_journal tools and register it at project scope [8][17].
4. Run claude mcp list, show the server and the environment-variable warning when SNOW_TOKEN is unset [8].
5. Ask Claude in the CLI to summarize INC0048211 and show the tools/call in the transcript [16].
6. Repeat the same call from a 20-line Agent SDK script with allowed_tools set to the two read-only tools [13].

Hands-on lab. Build and register a read-only ServiceNow MCP server and a read-only device MCP server and call them from an Agent SDK script.

1. Clone the course repository and run /init; edit CLAUDE.md to include the boundary policy.
2. Create .claude/settings.json with the deny rules supplied in the lab sheet.
3. Implement servicenow-ro with get_incident(number) and list_work_notes(sys_id) using GET on the Table API with sysparm_fields to limit fields [1][17].
4. Implement device-ro with show_virtual(device, name) and show_pool(device, name) backed by fixed command templates, not free text.
5. Register both servers with claude mcp add at project scope and commit .mcp.json without secrets [8].
6. Write triage_check.py with the Agent SDK, passing allowed_tools for the four tools only and removing Bash from tools [13].
7. Run the script against INC0048211 and save the transcript.

8. Search the transcript and .mcp.json for any password or token string and confirm there are none.

Lab success criteria.

- Both MCP servers appear in claude mcp list and respond to tools/list.
- The Agent SDK script reads the incident and one device object successfully.
- No credential appears in the repository, the transcript or tool output.
- An attempt to call Bash from the triage agent is refused or the tool is absent.

Deliverable. Working agent environment with scoped tools.

Common pitfalls.

- Printing debug output to stdout inside a stdio MCP server, which breaks JSON-RPC [17].
- Relying on CLAUDE.md to forbid an action instead of a deny rule or hook [12].
- Exposing a generic run_command tool on the device server; every tool must map to a fixed, read-only command template.
- Committing a real token in .mcp.json instead of ${VAR} expansion [8].
- Assuming a narrow allow rule overrides a broad deny; deny always wins [10].

Knowledge check.

1. Where do team-shared MCP server definitions live? Answer: In .mcp.json at the project root, created with claude mcp add --scope project [8].
2. What is the evaluation order of permission rules in Claude Code? Answer: Deny, then ask, then allow; the first match decides [10].
3. Why must a stdio MCP server never print to stdout? Answer: stdout carries the JSON-RPC messages, so stray output corrupts them [17].
4. How is an SDK MCP tool named in allowed_tools? Answer: mcp__{server_name}__{tool_name} [13].

Slides and assets to prepare.

- CLAUDE.md template for the triage repository.
- settings.json starter with deny and ask rules.
- snow_ro.py and device_ro.py starter servers.
- Diagram of Claude Code, Agent SDK, MCP servers and target systems with trust boundaries.
- Secrets handling checklist.

Sources for this module. 1, 8, 9, 10, 12, 13, 16, 17

Module 3: Incident intake through the Table API and the acknowledgement mail

Goal. Learners build an intake service that pulls incidents and their journal entries, validates them, lists the missing primary troubleshooting details and drafts an acknowledgement for a person to

send.

Outline topics.

- Querying incident and sys_journal_field through the ServiceNow Table API and validating records with Pydantic
- A primary troubleshooting checklist: affected VIP or service, symptom, start time, scope, last change, error text
- Claude compares the ticket against the checklist and lists exactly which details are missing
- Drafting the acknowledgement mail with the missing-details questions; a person reviews and sends

Learning objectives.

- Learners can query incident and sys_journal_field through the Table API with filters, field lists and paging.
- Learners can validate incoming records with Pydantic and reject malformed data.
- Learners can define a primary troubleshooting checklist and have Claude compare a ticket against it with structured output.
- Learners can post the drafted acknowledgement as a work note marked draft, never as a sent mail.

Querying incidents and journals

- List the queue with GET /api/now/table/incident?sysparm_query=assignment_group.name=lb-ops^active=true^ORDERBYDESCsys_updated_on [1].
- Limit payloads with sysparm_fields=number,sys_id,short_description,description,priority,cmdb_ci,opened_at and sysparm_exclude_reference_link=true [1].
- Page with sysparm_limit and sysparm_offset; the default limit is 10000, which is too large for an interactive tool [1].
- Read comments and work notes from sys_journal_field filtered by element_id=<incident sys_id> (and name=incident); confirm read access for the integration user on the PDI.
- Use sysparm_display_value=all when the agent needs both the display value and the stored value of choice fields such as state [1].

Validating records

- Model Incident and JournalEntry as Pydantic classes with required fields and explicit optional ones.
- Reject records that fail validation and log them; never pass unvalidated JSON to the model.
- Store the raw Table API response next to the parsed object so later citations can point to it.
- Normalize timestamps to UTC with the instance time zone recorded.

The primary troubleshooting checklist

- Fields: affected VIP or service, symptom, start time, scope (users, sites), last change, error text or screenshot.
- Each checklist item has a status of present, missing or ambiguous plus the ticket text that supports it.
- Define the checklist as a JSON schema and ask for structured output so the result is machine-checkable [14].
- Treat a success result without structured_output as a failure, as the SDK docs advise [14].
- Unknowns are allowed and must be explicit; the agent may not infer the VIP from the short description alone unless it quotes the text.

Drafting and posting the acknowledgement

- The draft contains the acknowledgement line, the missing-details questions and the expected next update time.
- Post the draft as a work note with a 'DRAFT - not sent' prefix via PATCH /api/now/table/incident/{sys_id} with a work_notes value [1].
- The engineer copies or sends the mail from ServiceNow; the reply returns through the watermark and inbound action [2].
- The intake tool records which tools were used (servicenow.read only) in the audit log.

Instructor demo.

1. Run the queue query with sysparm_query and sysparm_fields and show the JSON [1].
2. Fetch the journal entries for INC0048211 and show how the work notes and comments are separated.
3. Validate the record with Pydantic and deliberately break a field to show the ValidationError path.
4. Run the checklist agent with output_format and show the structured_output for vip, start_time, scope and last_change [14].
5. Post the draft acknowledgement as a work note and show it on the incident form [1].
6. Reply from a test mailbox and show the reply attached to the incident [2].

Hands-on lab. Ingest five synthetic incidents, generate the missing-details list and acknowledgement draft for each, and post it as a work note.

1. Implement fetch_queue() with sysparm_query, sysparm_fields and paging [1].
2. Implement fetch_journal(sys_id) for comments and work notes.
3. Write Pydantic models for Incident, JournalEntry and ChecklistResult.
4. Write the checklist JSON schema and the intake prompt; call the Agent SDK with output_format [14].
5. Handle error_max_structured_output_retries by flagging the incident for manual intake [14].
6. Generate the acknowledgement draft from the checklist result using a fixed template.
7. Post each draft as a work note prefixed DRAFT and capture the response sys_id [1].
8. Write an audit line for each incident with tools used and checklist outcome.

Lab success criteria.

- All five incidents are ingested and validated, or rejected with a logged reason.
- Each checklist result marks every item present, missing or ambiguous with quoted ticket text.
- Each draft asks only for details that are actually missing.
- No mail is sent by the tool; all drafts are work notes.

Deliverable. Intake service and acknowledgement mail generator.

Common pitfalls.

- Using sysparm_display_value=true and then writing the display value back into a choice field [1].
- Pulling the whole incident record with all reference links and blowing the context window; always set sysparm_fields [1].
- Letting the model guess the VIP from the CI name without quoting evidence.
- Posting to comments (customer-visible) instead of work_notes (internal).

Knowledge check.

1. Which parameter limits the fields the Table API returns? Answer: sysparm_fields, a comma-separated list [1].
2. How do you page through a large incident list? Answer: Increase sysparm_offset by sysparm_limit on each request [1].
3. What does the SDK return when the agent cannot satisfy your schema? Answer: A result with subtype error_max_structured_output_retries [14].
4. Why post the acknowledgement as a work note rather than send it? Answer: Sending mail to the caller is a person-only action; the work note is a reviewable draft.

Slides and assets to prepare.

- Checklist JSON schema and Pydantic models.
- Five synthetic incidents with deliberately missing details.
- Acknowledgement mail template.
- Slide: Table API query anatomy with sysparm parameters.

Sources for this module. 1, 2, 14

Day 2

Module 4: Evidence collection from load balancers and ACI

Goal. Learners write deterministic read-only collectors for F5, NetScaler, Radware Alteon, other load balancers and Cisco APIC that return typed evidence records and never infer anything without data.

Outline topics.

Gemnoor Tech
For more info - https://gemnoor.com

- F5: tmsh show ltm virtual and pool, iControl REST /mgmt/tm/ltm and /mgmt/tm/sys for stats and logs
- Citrix NetScaler NITRO API: lbvserver, servicegroup and HA node status; Radware Alteon and other load balancers (Anaya LB) through their CLI
- Cisco APIC REST API: fault, health score and endpoint lookups for the affected tenant
- Typed evidence records with device, timestamp, command and raw output; nothing inferred without data

Learning objectives.

- Learners can collect virtual server, pool and member status from F5 through tmsh and iControl REST.
- Learners can collect lbvserver statistics and HA status from NetScaler through NITRO and the CLI.
- Learners can capture Radware Alteon and other load balancer output through CLI adapters with recorded fixtures.
- Learners can query APIC faults, health scores and endpoints for a tenant through the REST API.
- Learners can define an evidence record that carries device, timestamp, command and raw output.

F5 BIG-IP

- Use tmsh show ltm virtual <name> and tmsh show ltm pool <name> members for status and statistics; the available tmsh commands depend on provisioned modules [4].
- iControl REST mirrors tmsh: /mgmt/tm/ltm/pool is the collection and /mgmt/tm/ltm/pool/~Common~<name> a single resource, keyed by name, partition and subPath [21].
- Pool members can be forced offline with state user-down or drained with session user-disabled; both are state-changing and excluded from collectors [18].
- show sys connection filters by ss-server-addr to list connections to a backend pool member; cs-server-addr is the client-side local (virtual) address [20].
- Monitor settings to capture: send, recv, interval (default 5 s) and timeout (default 16 s) [19].

Citrix NetScaler

- Read statistics with GET /nitro/v1/stat/lbvserver?args=name:<vserver> and config with GET /nitro/v1/config/lbvserver/<name> [22].
- Authenticate with a session cookie NITRO_AUTH_TOKEN or per-request X-NITRO-USER and X-NITRO-PASS headers, never both [22].
- HA status: show ha node returns Node State, Master State, Sync State and Propagation; a healthy secondary shows Sync State SUCCESS [23].
- The CLI reference is organized by feature area (lb, ssl, system, ha) for finding the matching show command [5].

Radware Alteon and other load balancers

- Alteon uses menu-path CLI commands such as /stats/slb/virt <x>, /stats/slb/group and /info/l3/ha [25].
- Radware publishes a Command Reference, CLI Application Guide, REST API User Guide and Troubleshooting Guide; obtain the version matching the lab appliance from the Radware portal [26].
- For 'Anaya LB' and any platform without a documented API, build a CLI capture adapter: fixed command list, prompt detection, timeout and raw output capture.
- Without a lab appliance, collectors run against recorded fixtures supplied by the client's engineers, with device names and IPs sanitized.

Cisco APIC

- Log in with POST /api/aaaLogin.json and reuse the APIC-cookie; the default session timeout is 600 seconds [24].
- Faults for a tenant: GET /api/mo/uni/tn-<tenant>.json?query-target=subtree&target-subtree-class=faultInst, or class queries on faultInst sorted with order-by=faultInst.severity|desc [24].
- Unhealthy objects: query-target-filter=lt(healthInst.cur,"50") [24].
- Endpoint lookup for the backend IP through the fvCEp class with a filter on ip.
- Every APIC call in the collector is a GET, which changes nothing [24].

The evidence record

- Fields: evidence_id, incident, device, platform, command_or_endpoint, collected_at (UTC), duration, exit status, raw_output, parsed summary.
- Raw output is kept verbatim; the parsed summary is produced by deterministic code, not by Claude.
- If a collector fails, the record says so; the agent must not fill the gap.
- Evidence ids are what Claude cites in the next module.

Instructor demo.

1. Run tmsh show ltm pool pool_portal members on the lab F5 and the equivalent GET on /mgmt/tm/ltm/pool/~Common~pool_portal/members [21].
2. Call GET /nitro/v1/stat/lbvserver?args=name:vs_portal on the NetScaler VPX and show the curstate field [22].
3. Run show ha node on the NetScaler pair and point out Sync State [23].
4. Log in to the DevNet ACI simulator and run a faultInst class query sorted by severity [24][27].
5. Play back a recorded Alteon /stats/slb/virt output through the capture adapter [25].
6. Show the resulting evidence bundle JSON for INC0048211.

Hands-on lab. Write deterministic collectors for each platform and run them against the lab devices for one incident.

1. Implement f5_collect(device, virtual) using iControl REST for virtual, pool and member status [21].

2. Implement ns_collect(device, vserver) using NITRO stat and config endpoints and show ha node [22][23].
3. Implement alteon_collect(fixture) parsing /stats/slb/virt and /stats/slb/group output [25].
4. Implement apic_collect(tenant, ip) for faults, health below 50 and endpoint lookup [24].
5. Wrap each collector so it returns an EvidenceRecord and catches timeouts.
6. Expose the collectors through the device-ro MCP server as fixed tools.
7. Run all collectors for INC0048211 and write the evidence bundle to runs/<incident>/evidence.json.
8. Diff a second run to confirm the collectors are repeatable apart from counters and timestamps.

Lab success criteria.

- Each platform produces at least one evidence record for the incident.
- Every record has device, timestamp, command or endpoint and raw output.
- No collector issues a POST, PUT, PATCH or DELETE except the APIC login.
- Failed collections are recorded as failures, not omitted.

Deliverable. Evidence bundle per incident.

Common pitfalls.

- Using cs-server-addr when you meant the pool member; for backend connections filter on ss-server-addr [20].
- Letting NITRO requests carry both the auth cookie and X-NITRO headers [22].
- Forgetting APIC token expiry after 600 seconds and silently returning empty results [24].
- Parsing raw CLI output with the model instead of deterministic parsers.
- Using an admin account for collectors because it was easier in the lab.

Knowledge check.

1. Which iControl REST path returns a single pool in the Common partition? Answer: /mgmt/tm/ltm/pool/~Common~<pool name> [21].
2. Which NetScaler output field shows whether HA configuration sync is working? Answer: Sync State in show ha node; SUCCESS on a healthy secondary [23].
3. How do you list only unhealthy APIC objects? Answer: query-target-filter=lt(healthInst.cur,"50") [24].
4. Why keep the raw output in the evidence record? Answer: So every claim Claude makes can be checked against the exact device response.

Slides and assets to prepare.

- Collector contract and EvidenceRecord schema.
- Command and endpoint cheat sheet per platform.
- Recorded Alteon and Anaya LB fixtures.
- Postman or curl collection for iControl REST, NITRO and APIC.

Gennoor Tech INNOVATE BUILD For more info - https://gennoor.com

Sources for this module. 4, 5, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27

Module 5: Claude suggests troubleshooting steps as structured JSON

Goal. Learners build a suggestion agent that takes a ticket and its evidence and returns ranked likely causes, each with a confirming command, expected result, command class and evidence citations.

Outline topics.

- Prompt design: ticket plus evidence in, ranked likely causes out, each with a confirming CLI command and expected result
- JSON schema for a suggested step: platform, command, class (read_only or state_changing), rationale, evidence citations
- Testing the prompt with missing, stale and contradictory evidence; unknowns must be marked
- Comparing suggestions against a vendor-documented playbook for the same symptom

Learning objectives.

- Learners can design a step schema that separates read-only from state-changing commands.
- Learners can prompt Claude to rank causes and cite evidence ids for each.
- Learners can test the agent with missing, stale and contradictory evidence.
- Learners can score suggestions against an engineer's notes and a vendor-documented check.

Prompt design

- Input: validated ticket, checklist result and evidence bundle; output: ranked causes with one confirming step each.
- Instruct Claude to cite evidence ids for every statement and to put anything unsupported in an unknowns list.
- Give platform context: for a pool member down, the confirming check is the monitor configuration and result (send, recv, interval, timeout) [19].
- Keep device commands as templates from the allowlist; Claude chooses a template and fills parameters, not free text.

The step schema

- Fields: rank, cause, platform, device, command_template, parameters, class (read_only or state_changing), expected_result, rationale, evidence_ids, confidence.
- Generate the schema from Pydantic with model_json_schema() and pass it as output_format [14].
- Schemas are validated as JSON Schema draft-07; keep them shallow to avoid retries [14].
- Example read-only step: tmsh show ltm monitor https https_portal; example state-changing step: modify ltm pool pool_portal members modify { web-04:443 { state user-down } } [18][19].

Testing with bad evidence

- Missing: remove the pool evidence; the agent must ask for it, not guess.

- Stale: evidence older than the incident start time must be flagged.
- Contradictory: F5 shows a member down while APIC shows the endpoint learned and healthy; the agent must present both and rank a check that resolves the conflict [24].
- Connectivity checks: extended ping and traceroute need privileged EXEC on Cisco IOS; the agent must not suggest them for a user-EXEC account [6].

Scoring against playbooks

- Score first-step accuracy: did the top-ranked step match what the engineer did first?
- Score citation validity: does every evidence id exist in the bundle?
- Compare with vendor-documented checks, for example the meaning of ! and . in ping output and * in traceroute [6].
- Keep a scored set of incidents as the prompt regression suite used in module 9.

Instructor demo.

1. Show the evidence bundle for INC0048211 and the step schema side by side.
2. Run the suggestion agent and show the structured_output with two ranked causes [14].
3. Delete the pool evidence and rerun; show the unknowns list growing.
4. Inject a contradictory APIC endpoint record and show the agent proposing a resolving check [24].
5. Score the run against the engineer's notes on screen.

Hands-on lab. Build the suggestion agent, run it on three incidents and score its first-step accuracy against the engineer's manual notes.

1. Write the Step and Suggestion Pydantic models.
2. Write the system prompt with citation and unknowns rules.
3. Call the Agent SDK with output_format and only the device-ro read tools available [13][14].
4. Run it on the VIP-down, member-flapping and HA-sync incidents.
5. Validate that every command_template exists in the allowlist and every evidence id exists.
6. Run the missing, stale and contradictory variants.
7. Score first-step accuracy and citation validity in the provided sheet.
8. Commit the three scored cases as regression fixtures.

Lab success criteria.

- Every suggestion validates against the schema.
- Every step is classed read_only or state_changing correctly.
- Every evidence citation resolves to a record in the bundle.
- In the missing-evidence case the agent lists the gap instead of inventing data.

Deliverable. Troubleshooting step suggestion agent.

Common pitfalls.

GenNoorTech TRAINING UNIT For more info: https://gennoor.com

- Letting Claude write free-form commands rather than choosing from templates.
- Trusting the model's own class label without checking it against the allowlist.
- Over-nested schemas that cause repeated validation retries [14].
- Suggesting privileged commands to an account that only has user EXEC access [6].

Knowledge check.

1. What two fields stop a suggested step from being run blindly? Answer: class (read_only or state_changing) and the evidence_ids that justify it.
2. How do you supply a Pydantic schema to the Agent SDK? Answer: output_format={'type': 'json_schema', 'schema': Model.model_json_schema()} [14].
3. Which F5 monitor settings explain a member marked down by an https monitor? Answer: send, recv, interval and timeout, with recv defining the expected response string [19].
4. What should the agent do when F5 and APIC disagree? Answer: Present both pieces of evidence and rank a check that resolves the conflict.

Slides and assets to prepare.

- Step schema reference card.
- Scoring sheet for first-step accuracy.
- Bad-evidence variants of three incidents.
- Slide: anatomy of a cited suggestion.

Sources for this module. 6, 13, 14, 18, 19, 24

Module 6: Guardrails, approvals and audit for incident automation

Goal. Learners enforce command classes with allowlists, permission rules and hooks, record approvals and write every action to an audit log and to ServiceNow work notes.

Outline topics.

- Command classification and allowlists: read-only commands may run in the lab, state-changing commands always wait
- Approval record: who approved, which command, which device, when, and the output captured
- Writing every action to an audit log and to ServiceNow work notes through the Table API
- Failure modes: wrong device, wrong incident, hallucinated command; detection and fail-closed behaviour

Learning objectives.

- Learners can build an allowlist that classifies each command template as read-only or state-changing.
- Learners can enforce the allowlist with deny rules and a PreToolUse hook that fails closed.

- Learners can create an approval record that captures approver, command, device, time and output.
- Learners can write audit entries to a local log and to the incident's work notes.

Command classification

- Every template is tagged read_only or state_changing in a versioned allowlist file; anything not on the list is denied.
- F5 examples: show ltm pool is read-only; modify ltm pool ... state user-down or session user-disabled are state-changing [18].
- delete sys connection is state-changing and, without a port, removes all connections matching the address [20].
- Tool annotations like readOnlyHint are hints; the allowlist is the source of truth [13][16].

Enforcement layers

- Permission rules: deny wins over ask and allow, so deny broad patterns first [10].
- A PreToolUse hook reads tool_name and tool_input from stdin, looks up the template class and exits 2 to block state-changing calls without an approval id [11].
- Set onFailure to block so a crashed or timed-out hook fails closed [11].
- In the SDK, hooks run before deny rules and the deny rules apply even in bypassPermissions mode; never run the triage agent in bypassPermissions [15].
- Use dontAsk mode for headless runs so anything not pre-approved is denied rather than prompting [15].

The approval record

- Fields: approval_id, incident, step id, command, device, requested_by, approved_by, approved_at, expiry, output_hash.
- An approval covers one command on one device for one incident and expires.
- The approver must not be the agent or the requesting session.
- MCP guidance says hosts should show tool inputs before invocation and ask for confirmation on sensitive operations; the approval UI shows the exact command [16].

Audit trail

- Write JSON lines for every tool call: who, what, when, inputs, outputs hash, decision.
- Mirror a human-readable summary to the incident's work_notes through PATCH on the Table API [1].
- MCP clients should log tool usage for audit purposes [16].
- Keep audit logs separate from the agent's working directory so the agent cannot edit them.

Failure modes

- Wrong device: the hook checks that the device belongs to the incident's CI.
- Wrong incident: approvals are bound to the incident sys_id.

- Hallucinated command: any template id not in the allowlist is denied.
- Hook errors: exit 1 is non-blocking, so policy hooks must use exit 2 or the deny decision [11].

Instructor demo.

1. Show the allowlist file with read_only and state_changing tags.
2. Show the PreToolUse hook script and its configuration in settings.json [11].
3. Ask the agent to force web-04 offline and show the hook blocking it with a reason [11].
4. Create an approval record in the approval tool and rerun; show the command passing.
5. Show the audit JSON lines and the work note on INC0048211 [1].
6. Kill the hook mid-run and show onFailure block keeping the call denied [11].

Hands-on lab. Add the approval gate and audit writer, then try to push a state-changing command through without approval and watch it refuse.

1. Create allowlist.yaml with every template from the collectors and suggestion agent, each classed.
2. Add deny rules for anything outside the device-ro and approved-exec tools [10].
3. Write the PreToolUse hook that checks class, approval id, device and incident [11].
4. Set onFailure block on the hook [11].
5. Implement approve(step_id, approver) that writes the approval record with expiry.
6. Implement audit_write() to JSONL and to work_notes [1].
7. Attempt the user-down command with no approval and capture the refusal.
8. Approve it and run it on the lab F5, then confirm the audit and work note entries.

Lab success criteria.

- The unapproved state-changing command is blocked and the block is logged.
- The approved command runs once and only on the approved device.
- Every tool call appears in the audit log with inputs and outcome.
- The work note on the incident shows the approval and the command output summary.

Deliverable. Approval gate and audit trail.

Common pitfalls.

- Writing the hook to exit 1 on a policy violation, which does not block [11].
- Running the agent in bypassPermissions to save time in the lab [15].
- Approvals with no expiry that can be replayed on another incident.
- Letting the agent write to the same file the audit trail lives in.

Knowledge check.

1. Which hook exit code blocks a tool call? Answer: Exit code 2, with stderr as the reason [11].

2. Does a PreToolUse hook that returns allow bypass deny rules? Answer: No; deny and ask rules are still evaluated [15].
3. What does delete sys connection do if no port is given? Answer: Deletes all connections that match just the IP address [20].
4. Name four fields of an approval record. Answer: Approver, command, device, timestamp (plus incident, expiry and output hash).

Slides and assets to prepare.

- allowlist.yaml template.
- PreToolUse hook script and settings snippet.
- Approval record schema.
- Audit log schema and sample.
- Slide: layers of defense (rules, hooks, approval, audit).

Sources for this module. 1, 10, 11, 13, 15, 16, 18, 20

Module 7: Running approved steps and reading the result

Goal. Learners execute approved commands through SSH and REST wrappers, let Claude interpret the result against the expected outcome and control the loop until the cause is confirmed or the incident is escalated.

Outline topics.

- Executing approved commands through SSH and REST wrappers with timeouts and output capture
- Claude interprets command output against the expected result and proposes the next step
- Loop control: maximum steps, stop conditions and escalation to a senior engineer
- ServiceNow Flow Designer trigger that starts the loop when an incident is assigned to the group

Learning objectives.

- Learners can wrap SSH and REST execution with timeouts, output capture and evidence records.
- Learners can have Claude compare actual output with the expected result and propose the next step.
- Learners can enforce loop limits, stop conditions and escalation.
- Learners can start the loop from a ServiceNow trigger when an incident is assigned to the group.

Execution wrappers

- SSH wrapper: fixed template, connect timeout, command timeout, prompt detection, raw output saved as evidence.

GenNoor Tech INNOVATE. BUILD. For more info: https://gennoor.com

- REST wrappers for iControl REST, NITRO and APIC reuse the collector auth and only call approved endpoints [21][22][24].
- Every execution produces an evidence record linked to the approval id.
- Read-only steps run automatically in the lab; state-changing ones only with a valid approval.

Interpreting results

- Claude receives expected_result, actual output and the step rationale and returns confirmed, refuted or inconclusive with citations.
- Ping and traceroute output must be read by its documented meaning: ! reply, . timeout, * no response before the timer [6].
- For NetScaler HA steps, Sync State values such as SUCCESS or AUTO DISABLED decide the next step [23].
- Each interpretation either proposes the next step or proposes stopping with a confirmed cause.

Loop control

- Maximum steps per incident (for example eight) and maximum elapsed time.
- Stop when a cause is confirmed with evidence, when two consecutive steps are inconclusive, or when the next step is state-changing and no approver responds.
- Escalate by assigning to a senior engineer and writing the evidence summary as a work note [1].
- Use the SDK permission mode that denies anything not pre-approved for unattended runs [15].

Starting from ServiceNow

- A record-based Flow Designer trigger on incident update, condition assignment group is lb-ops, calls the triage service [3].
- Flows can start with a delay because events are queued and processed in order [3].
- The flow passes only the incident sys_id; the service pulls everything else through the Table API [1].
- Guard against duplicate starts by recording a run id on the incident.

Instructor demo.

1. Approve and run tmsh show ltm monitor https https_portal through the SSH wrapper and show the evidence record.
2. Show Claude marking the monitor hypothesis confirmed with citations to the output.
3. Show the next suggested step being state-changing and waiting for approval.
4. Run an inconclusive branch twice and show automatic escalation with a work note.
5. Show the Flow Designer flow with a record-based trigger calling the service [3].

Hands-on lab. Run the suggest, approve, execute, interpret loop end to end on one incident until the cause is confirmed.

1. Implement ssh_exec(template, params, approval_id) with timeouts and capture.

2. Implement rest_exec for one approved endpoint per platform.
3. Implement interpret(step, output) with structured output confirmed, refuted or inconclusive [14].
4. Implement the loop with max steps, max time and stop conditions.
5. Implement escalate() that reassigns and writes a work note [1].
6. Build the Flow Designer flow that calls the service on assignment [3].
7. Assign INC0048211 to lb-ops and watch the loop start.
8. Approve the state-changing step when prompted and run to a confirmed cause.

Lab success criteria.

- The loop starts from the ServiceNow assignment.
- Read-only steps run automatically and state-changing steps wait for approval.
- The loop stops with a confirmed cause backed by cited evidence, or escalates.
- All executions are in the audit log and summarized in work notes.

Deliverable. Supervised troubleshooting loop.

Common pitfalls.

- No command timeout, so a hung SSH session blocks the loop.
- Feeding Claude only a parsed summary and losing the detail needed to interpret the result.
- Endless loops when the agent keeps proposing slightly different read-only checks.
- Assuming the flow fires instantly after assignment [3].

Knowledge check.

1. What are three stop conditions for the loop? Answer: Confirmed cause, repeated inconclusive results, or an unapproved state-changing next step (plus max steps or time).
2. What does a period mean in Cisco ping output? Answer: The server timed out waiting for a reply [6].
3. Which trigger type starts the flow on assignment? Answer: A record-based trigger on the incident table [3].
4. Why does the flow pass only the sys_id? Answer: So the service re-reads current data through the Table API rather than trusting a payload.

Slides and assets to prepare.

- Loop state diagram.
- SSH and REST wrapper starters.
- Flow Designer flow export for the lab.
- Escalation work note template.

Sources for this module. 1, 3, 6, 14, 15, 21, 22, 23, 24

Day 3

GenNoorTech PRIVATE LABS For more info: https://gennoor.com

Module 8: The reason, action and next action template

Goal. Learners generate, review and post a reason, action and next action resolution template from the evidence bundle and execution outputs, and create a knowledge article draft for reuse.

Outline topics.

- Template fields: reason (root cause with evidence), action (what was done, by whom, approved by whom), next action (owner and due date)
- Claude drafts the template from the evidence bundle and command outputs with a citation for every claim
- Posting resolution notes and close codes to the incident and creating a knowledge article draft
- Template variants per symptom family: VIP down, pool member flapping, HA sync failure, certificate expiry

Learning objectives.

- Learners can define the template fields and what evidence each requires.
- Learners can have Claude draft the template with a citation for every claim and explicit unverified items.
- Learners can post resolution notes and close code through the Table API after review.
- Learners can map the template to a KCS knowledge article and maintain variants per symptom family.

Template fields

- Reason: root cause with evidence ids, for example monitor recv mismatch on web-04 after a deploy.
- Action: what was done, by whom, approved by whom and when, from the approval and audit records.
- Next action: owner, task and due date; owners come from the hand-offs mapped in module 1.
- Unverified: anything Claude could not support with evidence, listed separately.

Drafting with citations

- Input is the evidence bundle, execution outputs and approvals only; no fresh device queries at this stage.
- Structured output enforces fields and a citations array per line [14].
- A deterministic check rejects any line whose citations do not resolve.
- The engineer edits and approves the draft before it is posted.

Posting to the incident

- PATCH /api/now/table/incident/{sys_id} with close_notes and close_code after approval; state change to Resolved is done by the engineer [1].
- Write the template summary to work_notes as well for the audit trail [1].
- Use the instance's own close code choices; read them rather than hard-coding.

GenNoorTech TRAINING INNOVATE BUILD For more info - https://gennoor.com

- If the PATCH fails validation (for example a mandatory close field), surface the instance error to the engineer instead of retrying with guessed values [1].

Knowledge article draft

- The KCS Article template has Issue, Environment, Cause and Resolution fields and maps cleanly to the template [7].
- Predefined templates are inactive by default and must be activated and linked to a knowledge base [7].
- The feature needs the Knowledge Management Advanced plugin; check it on the PDI [7].
- Create the article as a draft through the Table API on the knowledge table; publishing stays with the knowledge owner.

Variants per symptom family

- VIP down: virtual status, pool availability, address and route checks.
- Pool member flapping: monitor settings (interval default 5 s, timeout 16 s) and recv strings [19].
- HA sync failure: Sync State and Propagation on NetScaler show ha node [23].
- Certificate expiry: certificate object and expiry date evidence, linked to the Certificate Renewal course.

Instructor demo.

1. Show the evidence bundle and approvals for the resolved incident.
2. Generate the template and show each line's citations [14].
3. Break a citation and show the deterministic check rejecting the draft.
4. Post close_notes and close_code after review and show the incident [1].
5. Create a KCS-style knowledge draft and show the Issue, Environment, Cause and Resolution fields [7].

Hands-on lab. Generate the template for the resolved incident, review it, post it to ServiceNow and create the knowledge article.

1. Write the ResolutionTemplate Pydantic model with per-line citations.
2. Generate the draft from the module 7 run [14].
3. Run the citation resolver and fix any rejected lines.
4. Review the draft and record your approval.
5. Post close_notes, close_code and a work note through the Table API [1].
6. Activate the KCS template on the PDI if needed and create the article draft [7].
7. Write one variant prompt for the HA sync failure family and test it on that incident [23].

Lab success criteria.

- Every reason and action line cites evidence that exists.
- Unverified items are listed, not hidden.

- Close notes and work note are on the incident; the state change was made by a person.
- A knowledge article draft exists with all four KCS fields filled.

Deliverable. Reason, action and next action generator.

Common pitfalls.

- Letting the template agent query devices again and introduce new, unreviewed evidence.
- Hard-coding close codes that do not exist on the instance.
- Not activating the KCS template and then wondering why it is not offered [7].
- Writing next actions without an owner or date.

Knowledge check.

1. Which fields does the KCS Article template provide? Answer: Issue, Environment, Cause and Resolution [7].
2. Who changes the incident state to Resolved? Answer: The engineer, after reviewing the posted template.
3. What happens to a template line with an unresolvable citation? Answer: The deterministic check rejects the draft until it is fixed or moved to unverified.
4. What evidence anchors the HA sync failure variant? Answer: Sync State and Propagation from show ha node [23].

Slides and assets to prepare.

- Reason, action and next action template with field rules.
- Citation resolver script.
- KCS article mapping sheet.
- Four symptom-family variant prompts.

Sources for this module. 1, 7, 14, 19, 23

Module 9: The troubleshooting page: service, tests and delivery

Goal. Learners assemble a FastAPI troubleshooting page, test it with pytest against recorded device output and ship it through reviewed pull requests with GitHub Actions.

Outline topics.

- FastAPI troubleshooting page: incident queue, evidence view, suggested steps, approve and reject buttons, audit tab
- pytest suites for intake, classification and template generation with recorded device output as fixtures
- GitHub pull requests and Actions: lint, tests and a prompt regression check on every change
- Operating the page: runbook, on-call handover and what to do when the agent is wrong

Learning objectives.

GenMoor Tech TRAINING INNOVATION BUILD
For more info - https://genmoor.com

- Learners can build the page views: queue, evidence, suggested steps, approve and reject, audit.
- Learners can write pytest suites with recorded fixtures for intake, classification and template generation.
- Learners can add a prompt regression check to CI.
- Learners can write the runbook and on-call handover for the page.

FastAPI service

- Endpoints: GET /incidents, GET /incidents/{number}/evidence, GET /incidents/{number}/steps, POST /steps/{id}/approve, POST /steps/{id}/reject, GET /audit.
- Approve and reject call the approval service from module 6; the page never calls devices directly.
- Authenticate engineers and record the approver identity from the session, not from a form field.
- Read incidents through the Table API with the same read-only integration user [1].

Testing with recorded output

- Store recorded device output (F5, NetScaler, Alteon, APIC) as fixtures and load them with pytest fixtures and conftest.py.
- Test intake: checklist results for the five synthetic incidents.
- Test classification: every allowlist template classed correctly; unknown templates denied.
- Test the hook: a state-changing call without approval exits 2 [11].
- Use FastAPI's TestClient for endpoint tests.

Pull requests and Actions

- Workflow on pull_request: lint, unit tests, hook tests and the prompt regression suite.
- Prompt regression: the scored incidents from module 5 must keep first-step accuracy and citation validity at or above the baseline.
- Protect the main branch with required reviews and status checks.
- Secrets for live checks come from repository secrets, never fixtures.

Operating the page

- Runbook: start and stop, credentials rotation, PDI or instance outage, device unreachable.
- On-call handover: open incidents in the loop, pending approvals, last escalations.
- When the agent is wrong: reject the step, record the reason, add the case to the regression set.
- Review the audit log weekly for denied calls and repeated rejections.

Instructor demo.

1. Run the FastAPI app and walk through the queue, evidence, steps and audit tabs.
2. Approve a step from the page and show the audit entry.
3. Run pytest with recorded fixtures and show the hook test [11].
4. Open a pull request that degrades the prompt and show the regression check failing.

5. Fix it and merge with Actions green.

Hands-on lab. Assemble the page, write the regression suite and merge through a reviewed pull request with Actions green.

1. Implement the six endpoints and minimal HTML views.
2. Wire approve and reject to the approval service.
3. Write fixtures from recorded device output.
4. Write tests for intake, classification, hook and template generation.
5. Write the prompt regression test against the scored incidents.
6. Add the GitHub Actions workflow for pull requests.
7. Open a pull request, get a review from a partner and merge with checks green.
8. Write the runbook and handover template.

Lab success criteria.

- All endpoints work against the lab data.
- The suite covers intake, classification, hook and template generation and passes.
- The prompt regression check runs on every pull request.
- The runbook covers outage, wrong-agent and credential-rotation cases.

Deliverable. Tested troubleshooting page and runbook.

Common pitfalls.

- Letting the page call devices directly and bypass the approval service.
- Taking the approver name from a form field.
- Fixtures that contain real hostnames, IPs or secrets.
- Prompt changes merged without the regression check.

Knowledge check.

1. Why does the page never call devices directly? Answer: All device actions go through the approval and audit services so the guardrails cannot be bypassed.
2. What does the prompt regression check measure? Answer: First-step accuracy and citation validity on the scored incidents.
3. What should an engineer do when the agent is wrong? Answer: Reject the step with a reason and add the case to the regression set.
4. Which test proves the guardrail still works? Answer: The hook test that expects exit code 2 for an unapproved state-changing call [11].

Slides and assets to prepare.

- FastAPI starter app.
- pytest fixture pack of recorded device output.
- GitHub Actions workflow file.
- Runbook and handover templates.

Gennoor Tech
For more info - https://gennoor.com

Sources for this module. 1, 11

Module 10: Capstone: incident to resolution note, end to end

Goal. Teams take a new incident from acknowledgement to posted resolution note using the full workflow and defend it against the acceptance criteria.

Outline topics.

- A new incident lands in ServiceNow; the team runs the full workflow from acknowledgement to posted template
- Acceptance criteria review: no device change without approval, every claim cited, every action logged
- Demo and limitations statement

Learning objectives.

- Learners can run the full workflow on an unseen incident.
- Learners can show that no device changed without approval and every claim is cited.
- Learners can present the limitations of their automation honestly.

Capstone scenario

- A new incident lands in the PDI with a symptom the teams have not seen (for example an HA sync failure on the NetScaler pair after a change) [23].
- Teams run intake, acknowledgement draft, evidence collection, suggestions, approvals, execution and template.
- The instructor plays caller and senior approver.
- The capstone incident deliberately omits two checklist details so the acknowledgement step is exercised.
- Teams must use only the collectors and allowlist built during the course; no ad hoc device logins.

Acceptance criteria

- No state-changing command without an approval record.
- Every reason and action line cites evidence.
- Every tool call in the audit log and summarized in work notes [1].
- Resolution posted and knowledge draft created [7].

Demo and limitations

- Five-minute demo of the run.
- Limitations statement: platforms covered, known gaps, false-positive and false-negative cases seen.
- Next steps for production: credentials, change control, rollout.
- Show the audit log and the matching approval records during the demo, not only the final note.

Instructor demo.

1. Inject the capstone incident into the PDI and show it arriving in the lb-ops queue.
2. Show the acceptance checklist.
3. Walk through one prior run on the troubleshooting page as a model of a good demo.
4. Demonstrate how the audit log will be checked during review.

Hands-on lab. Capstone build and demo.

1. Pick up the capstone incident from the queue.
2. Draft and send (as a person) the acknowledgement.
3. Collect evidence from every relevant platform.
4. Run the suggestion agent and review the ranked steps.
5. Approve and execute the steps needed to confirm the cause.
6. Generate, review and post the reason, action and next action template.
7. Create the knowledge article draft.
8. Present the run and the limitations statement.

Lab success criteria.

- The incident is resolved with a posted template.
- Audit log shows zero unapproved state-changing commands.
- All template lines are cited.
- Limitations statement is specific and honest.

Deliverable. End-to-end troubleshooting automation and runbook.

Common pitfalls.

- Skipping the acknowledgement step under time pressure.
- Approving your own team's state-changing step without checking the device.
- Presenting only the happy path.

Knowledge check.

1. How do you prove no device changed without approval? Answer: Match every state-changing execution in the audit log to a valid approval record.
2. What belongs in the limitations statement? Answer: Platforms and symptoms covered, known gaps, and observed wrong suggestions.
3. What is the last action a person takes on the incident? Answer: Reviewing the posted template and changing the state to Resolved.

Slides and assets to prepare.

- Capstone incident pack.
- Acceptance checklist.
- Demo scoring rubric.

GenNoor Tech TRAIN. INNOVATE. BUILD. For more info: https://gennoor.com

Sources for this module. 1, 7, 23

6. Capstone plan

A repeatable troubleshooting workflow that picks up a ServiceNow incident, asks for the missing primary troubleshooting details, collects evidence from F5, NetScaler, Radware, other load balancers (Anaya LB) and Cisco ACI, suggests ranked CLI steps, runs only approved commands, and posts a reason, action and next action template with a full audit trail.

Teams receive an unseen ServiceNow incident and run the full workflow: intake and acknowledgement draft, evidence collection from every relevant platform, ranked cited suggestions, approved execution, and a posted reason, action and next action template with a knowledge draft, all visible on their troubleshooting page.

Milestones

1. Incident picked up, checklist and acknowledgement draft posted as a work note.
2. Evidence bundle collected and reviewed.
3. Suggestions generated; read-only steps executed; state-changing step approved and executed.
4. Cause confirmed or escalation made with evidence.
5. Template reviewed and posted; knowledge article draft created.
6. Demo and limitations statement presented.

Evaluation notes

- Zero state-changing executions without a matching approval record (pass/fail).
- Citation validity: every reason and action line resolves to evidence.
- Quality of the acknowledgement: asks only for genuinely missing details.
- Audit completeness: every tool call logged and summarized in work notes.
- Honesty and specificity of the limitations statement.

Assessment weights

Gennoor Tech
For more info - https://gennoor.com

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. An incident arriving after hours, an acknowledgement followed by questions about the VIP, start time and last change, and an engineer retyping the same first commands is a teaching scenario drawn from the client's description of the current process. The mechanics it relies on are real: ServiceNow notifications carry a watermark so replies are matched back to the incident [2], and incidents are read and updated through the Table API [1].

GenNoorTech TRAINING INNOVATE BUILD
For more info - https://gennoor.com

Claim or on-screen text	Basis
10 modules that run / every module ends with something you can run and review	Matches the course outline in content.py: ten modules, each with a lab and deliverable.
3 days, hands-on	Matches the outline (days 1 to 3).
4 device platforms in the lab	Course design: F5, NetScaler, Radware Alteon and Cisco ACI. Note that Radware may be fixture-based if no Alteon VA license is available [25][26], and the outline also names 'Anaya LB', which is fixture-only.
0 device changes without approval	A design goal enforced in the labs by deny rules, a PreToolUse hook that exits 2 and approval records [10][11]; it is a property of the course build, not a product guarantee.
Read-only tools by default; credentials stay in the vault, never in a prompt	Supported by scoped MCP tools and allowed_tools [13] and ${VAR} expansion in .mcp.json [8].
Pull the incident through the ServiceNow Table API	Table API GET on /api/now/table/incident [1].
Terminal: 'triage intake INC0048211 ... posted as work note (draft) ... audit: intake#1 tools used: servicenow.read'	Teaching scenario output from the course's own intake tool; the incident number and tool names are invented for the lab.
Terminal: 'tmsh show ltm virtual vs_portal_443 ... tmsh show ltm pool pool_portal members: 3/4 up web-04:443 DOWN'	Real tmsh commands [4][18]; device, object names and numbers are a teaching scenario.
Suggestion 1: 'tmsh show ltm monitor https https_portal'	The https monitor and its recv string are documented [19]; the specific object is a teaching scenario.
Suggestion 2: 'tmsh show sys connection cs-server-addr 10.20.4.14' to check backend resets	The command exists, but cs-server-addr is the client-side local (virtual) address; to see connections to pool member 10.20.4.14 the filter should be ss-server-addr [20]. Recommend correcting the on-screen line to ss-server-addr.
'modify ltm pool pool_portal members modify { web-04:443 {state user-down} }' is state-changing and blocked until approved	state user-down forces a member offline, overriding monitors [18]; blocking it until approval is the course guardrail [11].
'monitor https_portal: recv string mismatch on web-04:443'	Teaching scenario; consistent with how recv strings determine monitor success [19].
Posted resolution notes and KB draft KB0012077	Close notes via the Table API [1] and KCS-style knowledge templates [7]; numbers are invented.
'pytest 118 passed, prompt regression 12/12, Actions: green'	Illustrative output of the capstone build; not a measured result.

GemmaTech
For more info - https://gemmaai.com

ServiceNow Flow Designer trigger starts the loop when an incident is assigned	Record-based triggers on create or update are supported, with possible queue delay [3].
Every lab runs against a ServiceNow developer instance, lab load balancers and a Cisco ACI sandbox	Free PDIs [28], F5 VE trial [29], NetScaler VPX Express [30] and the DevNet ACI simulator [27].

References

[1] Table API. ServiceNow Product Documentation. https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html
Supports: Incident intake; guardrails and audit; reason, action and next action template. The Table API exposes /api/now/table/{tableName} (GET list, POST insert one record; multiple-record insertion is not supported) and /api/now/table/{tableName}/{sys_id} (GET, PATCH, PUT, DELETE, with DELETE returning 204 and no body); api_version is optional. GET supports sysparm_query (encoded query with =, !=, LIKE, STARTSWITH, ENDSWITH, ^ and ^OR, ORDERBY/ORDERBYDESC), sysparm_fields, sysparm_limit (default 10000), sysparm_offset, sysparm_display_value (true, false, all; default false) and sysparm_exclude_reference_link.

[2] Email and SMS notifications. ServiceNow Product Documentation.
https://www.servicenow.com/docs/bundle/zurich-platform-administration/page/administer/notification/concept/c_EmailNotifications.html
Supports: The troubleshooting workflow and the incident lifecycle; acknowledgement mail. A watermark is generated by default at the bottom of each notification email to allow matching incoming email to existing records; inbound email actions script how the instance responds to inbound email; a digest is a single email that summarizes activity for a selected notification over a time interval.

[3] Architecture Overview (Flow Designer / Workflow Studio). ServiceNow Product Documentation.
https://www.servicenow.com/docs/bundle/washingtondc-build-workflows/page/administer/flow-designer/concept/flow-designer-arch-overview.html
Supports: Running approved steps: Flow Designer trigger. Flows start from record-based (created, updated, deleted), schedule-based or application-based triggers; a scheduler works through the event queue in the order items were added, so a flow may not start immediately; process plans are compiled just in time and a cached copy is reused when nothing changed. The page is now served under the Workflow Studio name.

[4] F5 TMSH Reference - 17.x. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/
Supports: Evidence collection from load balancers and ACI. The TMSH reference is the collection of BIG-IP tmsh man pages organized into General, Commands and Modules sections, and the available man pages depend on which modules are provisioned on the BIG-IP system.

[5] ADC CLI Commands 14.1. NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-command-reference-int/current-release.html
Supports: Evidence collection from load balancers and ACI. The NetScaler ADC CLI command reference (14.1) is indexed by feature area such as lb, ssl, system and ha, each with a commands page and object sub-pages.

[6] Understand the Extended Ping and Extended Traceroute Commands. Cisco.
https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13730-ext-ping-trace.html
Supports: Claude suggests troubleshooting steps; running approved steps. Extended ping and extended traceroute work only in privileged EXEC mode, while normal ping and traceroute work in user and privileged EXEC; in ping output each ! is a reply and . is a timeout; traceroute prints * when the timer expires before a response.

[7] Knowledge article templates. ServiceNow Product Documentation.
https://www.servicenow.com/docs/r/washingtondc/servicenow-platform/knowledge-management/knowledge-

article-templates.html
Supports: The reason, action and next action template. ServiceNow provides Standard, FAQ, How To, What Is and KCS Article templates; the KCS Article template has Issue, Environment, Cause and Resolution fields (table kb_template_kcs_article); all predefined templates are inactive by default and only active templates associated with a knowledge base are shown; the feature is activated with the Knowledge Management Advanced plugin.

[8] Connect Claude Code to tools via MCP. Anthropic (Claude Code docs). https://code.claude.com/docs/en/mcp
Supports: Setting up Claude Code, the Agent SDK and MCP. claude mcp add --transport http <name> <url> adds a remote server (recommended), stdio servers are added with everything after -- passed to the process (for example claude mcp add --env KEY=value --transport stdio name -- cmd), and SSE is deprecated. Scopes are local (default, ~/.claude.json), project (.mcp.json in the project root, shared via version control, approval required) and user; precedence is local, project, user, plugin, then claude.ai connectors. .mcp.json supports ${VAR} and ${VAR:-default} expansion in command, args, env, url and headers.

[9] What is the Model Context Protocol (MCP)?. Model Context Protocol (modelcontextprotocol.io). https://modelcontextprotocol.io/introduction
Supports: Setting up Claude Code, the Agent SDK and MCP. MCP is an open-source standard for connecting AI applications to external systems (data sources, tools and workflows), described as a USB-C port for AI applications and supported across many clients so a server can be built once and integrated everywhere.

[10] Configure permissions. Anthropic (Claude Code docs). https://code.claude.com/docs/en/permissions
Supports: Setting up Claude Code; guardrails, approvals and audit. Permission rules are evaluated deny, then ask, then allow, and the first match wins regardless of specificity, so a broad deny such as Bash(aws *) cannot be carved out by a narrower allow. Rules live in .claude/settings.json (project), .claude/settings.local.json (local) and ~/.claude/settings.json (user). Deny and ask rules match any subcommand of a compound command, but Bash rules match the command text as written and are not a security boundary around a program.

[11] Hooks reference. Anthropic (Claude Code docs). https://code.claude.com/docs/en/hooks
Supports: Guardrails, approvals and audit for incident automation. A PreToolUse hook runs before a tool call and can block it; it receives JSON on stdin with tool_name, tool_input and tool_use_id; it blocks by exiting with code 2 (stderr becomes the reason) or by printing hookSpecificOutput with permissionDecision deny. Exit code 1 is a non-blocking error, silence does not approve, and a timed-out hook does not block unless onFailure is set to block.

[12] How Claude remembers your project. Anthropic (Claude Code docs). https://code.claude.com/docs/en/memory
Supports: Setting up Claude Code, the Agent SDK and MCP. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md and are shared through version control; /init generates a starting file; files can import others with @path syntax; CLAUDE.md is context, not enforced configuration, so blocking an action requires a PreToolUse hook. The page recommends keeping each CLAUDE.md under 200 lines.

[13] Give Claude custom tools (Agent SDK). Anthropic (Claude Code docs). https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Setting up the Agent SDK; evidence collection tools. In Python, tools are defined with the @tool decorator (name, description, input schema, async handler) and wrapped with create_sdk_mcp_server; they are passed in ClaudeAgentOptions(mcp_servers=...) and named mcp__{server}__{tool} in allowed_tools. Handlers return content blocks and can set is_error; annotations such as readOnlyHint are metadata, not enforcement. The tools option controls which built-ins are available, while allowed_tools controls auto-approval.

[14] Get structured output from agents. Anthropic (Claude Code docs). https://code.claude.com/docs/en/agent-sdk/structured-outputs
Supports: Claude suggests troubleshooting steps as structured JSON. Passing output_format={'type': 'json_schema', 'schema': ...} to query() returns validated data in ResultMessage.structured_output; Pydantic models can supply the schema via model_json_schema(); the SDK validates against JSON Schema draft-07 and re-prompts on mismatch, and failure ends with subtype error_max_structured_output_retries. A success result without structured_output must also be treated as a failure.

[15] Configure permissions (Agent SDK). Anthropic (Claude Code docs). https://code.claude.com/docs/en/agent-sdk/permissions
Supports: Guardrails, approvals and audit; running approved steps. The SDK evaluates hooks, deny rules, ask rules, permission mode, allow rules and finally the canUseTool callback; a deny rule applies even in bypassPermissions mode; dontAsk mode denies anything that would prompt; allowed_tools does not constrain bypassPermissions; and checks that must run on every call belong in a PreToolUse hook.

[16] Tools (MCP specification 2025-06-18). Model Context Protocol (modelcontextprotocol.io). https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Setting up MCP; guardrails. Clients discover tools with tools/list and invoke them with tools/call; a tool has name, description, inputSchema, optional outputSchema and annotations; execution errors are returned with isError true; there should always be a human in the loop able to deny invocations; clients must treat annotations as untrusted unless the server is trusted, and should log tool usage for audit.

[17] Build an MCP server. Model Context Protocol (modelcontextprotocol.io). https://modelcontextprotocol.io/docs/develop/build-server
Supports: Setting up MCP: building the read-only ServiceNow and device servers. The Python quickstart creates a server with from mcp.server import MCPServer; mcp = MCPServer('weather'), declares tools with @mcp.tool() using type hints and docstrings, and runs it with mcp.run(transport='stdio'); stdio servers must never write to stdout because it corrupts JSON-RPC messages, so logging goes to stderr.

[18] ltm pool - F5 TMSH Reference. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_pool.html
Supports: Evidence collection; guardrails (state-changing commands). Pool member options include state user-up | user-down (user-down forces the member offline, overriding monitors) and session user-enabled | user-disabled (used to drain connections gently); the member monitor defaults to the pool's monitors; load-balancing-mode defaults to round-robin; priority-group ranges 0 to 65535; example syntax: modify pool mypool members modify { pool_member_1:80 { monitor http } }.

[19] ltm monitor https - F5 TMSH Reference. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_monitor_https.html
Supports: Claude suggests troubleshooting steps; reason template variants. The https monitor's send string defaults to GET /, recv (the string the monitor looks for in the response) defaults to none, recv-disable requires recv, interval defaults to 5 seconds and timeout defaults to 16 seconds.

[20] sys connection - F5 TMSH Reference. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/sys/sys_connection.html
Supports: Evidence collection; promo claims check. show sys connection filters active connections by cs-client-addr (client-side remote), cs-server-addr (client-side local, i.e. the virtual address), ss-client-addr (server-side local) and ss-server-addr (server-side remote, i.e. the pool member); delete connection uses the same filters, and if no port is given the system deletes all connections matching the address.

[21] APIRef_tm_ltm_pool (iControl REST). F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_ltm_pool.html
Supports: Evidence collection from load balancers and ACI. The iControl REST collection URI is /mgmt/tm/ltm/pool (OPTIONS, GET) and the resource URI is /mgmt/tm/ltm/pool/~resource id (OPTIONS, GET, PUT, PATCH, DELETE, POST), with natural key name, partition and subPath; properties mirror tmsh, for example allowNat defaults to yes.

[22] lbvserver statistics (NITRO API 13.1). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/13-1/statistics/lb/lbvserver
Supports: Evidence collection from load balancers and ACI. Load balancing virtual server statistics are read with GET /nitro/v1/stat/lbvserver (optionally ?args=name:<name> or ?attrs=...), with headers Cookie: NITRO_AUTH_TOKEN=<token> and Accept: application/json; per-request authentication can instead use X-NITRO-USER and X-NITRO-PASS headers without the cookie.

[23] Upgrade a high availability pair. NetScaler (docs.netscaler.com). https://docs.netscaler.com/us/en/citrix-adc/current-release/upgrade-downgrade-citrix-adc-appliance/upgrade-downgrade-ha-pair.html

Supports: Evidence collection (HA status); reason template variant HA sync failure. show ha node prints Node State, Master State, INC State, Sync State and Propagation per node; in a healthy pair the primary shows Sync State ENABLED and the secondary SUCCESS, while during an upgrade the secondary shows Sync State AUTO DISABLED and Propagation AUTO DISABLED; force failover switches roles.

[24] Cisco APIC REST API Configuration Guide - Using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-rest-api-configuration-guide/cisco-apic-rest-api-configuration-guide/m_using_the_rest_api.html
Supports: Evidence collection from load balancers and ACI. APIC objects are read at /api/mo/{dn}.{json|xml} and classes at /api/class/{className}; aaaLogin (POST) returns a token used as the APIC-cookie, with a default session timeout of 600 seconds; query-target takes self (default), children or subtree; query-target-filter supports operators such as eq, wcard and and, for example lt(healthInst.cur,"50"); rsp-subtree-include can add faults or health; results page with page and page-size and sort with order-by=faultInst.severity|desc; GET makes no changes.

[25] Alteon 31.0.11.0 Release Notes - Maintenance fixes. Radware. https://portals.radware.com/releasenotes/Alteon_Release_Notes_31.0.11.0/Alteon_Release_Notes/AlteonOS-31_0_11_0-RN.1.41.html
Supports: Evidence collection: Radware Alteon CLI. Radware's own fix list references the Alteon CLI menu paths /stats/slb/virt <x> (virtual server statistics), /stats/slb/group (real server group statistics), /info/l3/ha (HA information), /cfg/slb/virt <vsid>/service <vport> and the diff flash command, which confirms the menu-path style an Alteon collector must handle.

[26] Alteon 31.0.11.0 Release Notes - Related documentation. Radware. https://portals.radware.com/releasenotes/Alteon_Release_Notes_31.0.11.0/Alteon_Release_Notes/AlteonOS-31_0_11_0-RN.1.51.html
Supports: Lab environment; evidence collection: Radware Alteon. The Alteon documentation set includes the Alteon VA Installation and Maintenance Guide, the Command Line Interface Application Guide, the Command Reference, the REST API User Guide and the Alteon Troubleshooting Guide.

[27] ACI Sandboxes. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment. Cisco DevNet offers an always-on ACI Simulator sandbox (shared, no reservation required) and a reservable ACI Simulator sandbox (up to 5 consecutive days) for APIC REST API requests over HTTP/HTTPS with XML and JSON.

[28] Understanding PDIs. ServiceNow Product Documentation. https://www.servicenow.com/docs/r/SmYvGPN0OygSQs53fsexIQ/yG_RdLKb649OcfnDiGEkwQ
Supports: Lab environment. Personal Developer Instances are free; a PDI unused for ten days is returned to the pool, and an idle PDI hibernates (database and application server shut down, data kept) until the owner signs in to the Developer Site, which restarts the ten-day countdown.

[29] Try BIG-IP and BIG-IQ Virtual Editions. F5. https://www.f5.com/trials/big-ip-virtual-edition
Supports: Lab environment. F5 offers a 30-day free trial of BIG-IP Virtual Edition: log in or register at MyF5, choose Trials under My Products & Plans, select the BIG-IP Virtual Edition & BIG-IQ Virtual Edition tile and request trial keys.

[30] Licensing (NetScaler VPX Express). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/licensing.html
Supports: Lab environment. NetScaler VPX Express for on-premises and cloud deployments needs no license file and offers 20 Mbps bandwidth, a maximum of 250 SSL sessions and 20 Mbps SSL throughput; from release 14.1 build 29.65 it includes the advanced features of the Premium license.

For more info - https://gennoor.com